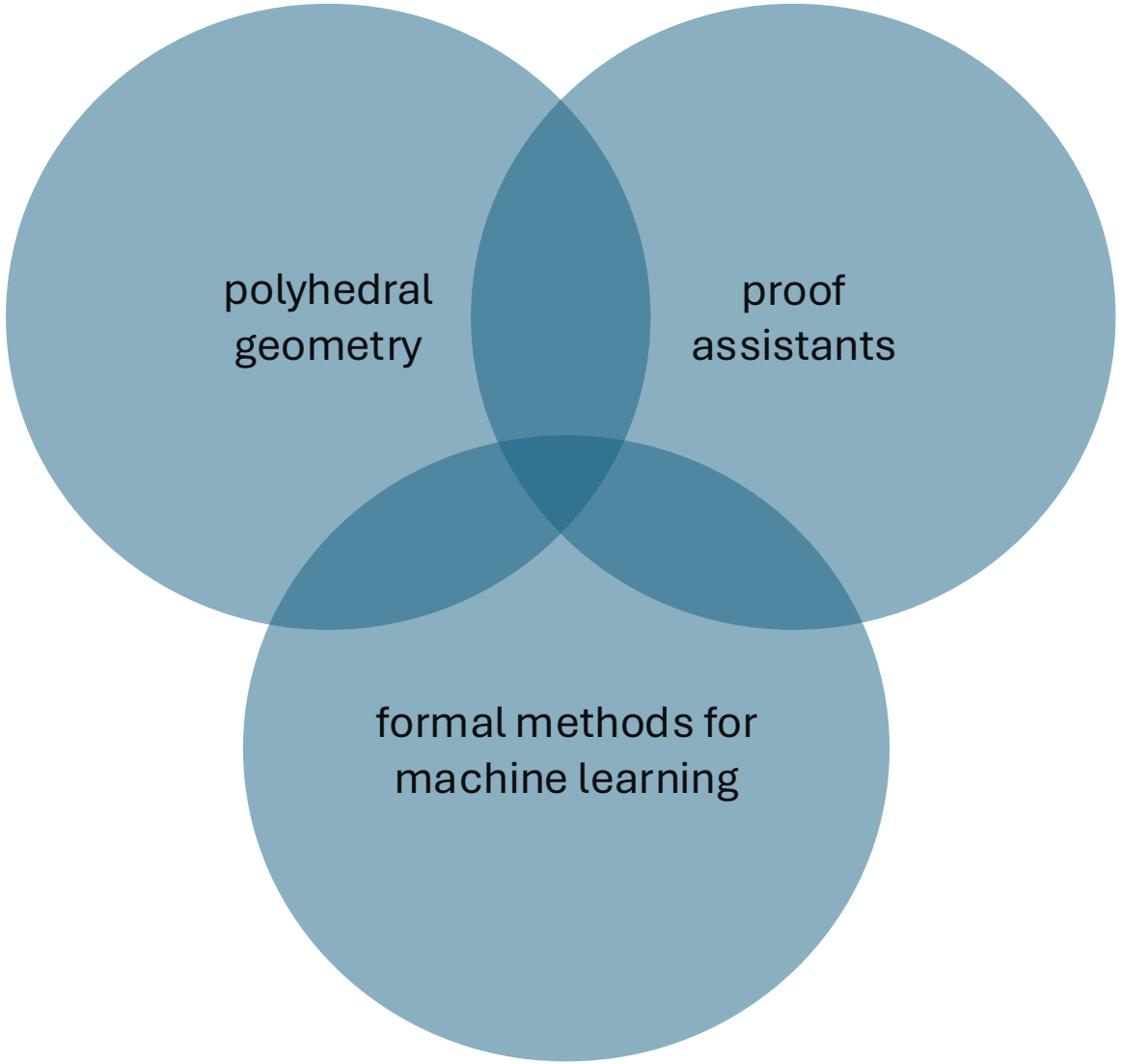
Polyhedra at Work: Neural Network Verification in Rocq

Kim Völlinger (TU Berlin)

Joint work: Andrei Aleksandrov, Leo Gummersbach, Malte Jackisch

Neural Network Verification in Rocq



Neural Networks: Why and what to verify?

US hospital: Racial Bias in Judgement of Health Care Needs (2019)

Tesla Autopilot (2015-2024)

at least 28 fatal accidents and 100 harmful accidents

https://link.springer.com/chapter/10.1007/978-3-031-44127-1_13

E.g. *robustness* is a desired property

Neural Network Verification in a Nutshell

I Model

- often real-valued ReLU networks

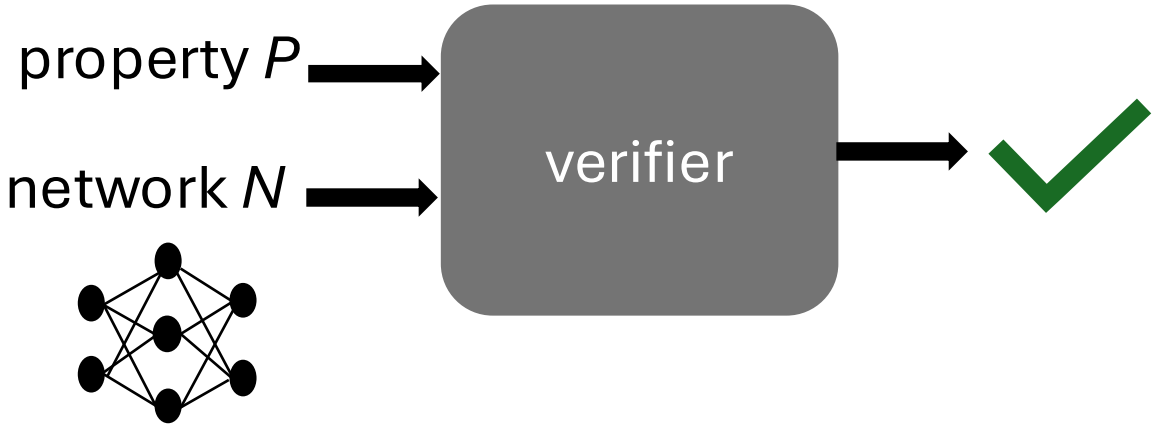
II Specification

- e.g. local robustness

III Verification method

- SMT solving, LP, abstract interpretation
- algorithmic soundness, sometimes completeness
- heuristics
- large unverified software tool

Exploiting *Verified* Networks



But:
N does not satisfy *P* under some circumstances
[Zombori et al., ‘21] [Jia & Rinard, ‘21] [Szász et al., ‘25] [Cordeiro et al., ‘25]

Soundness Gap

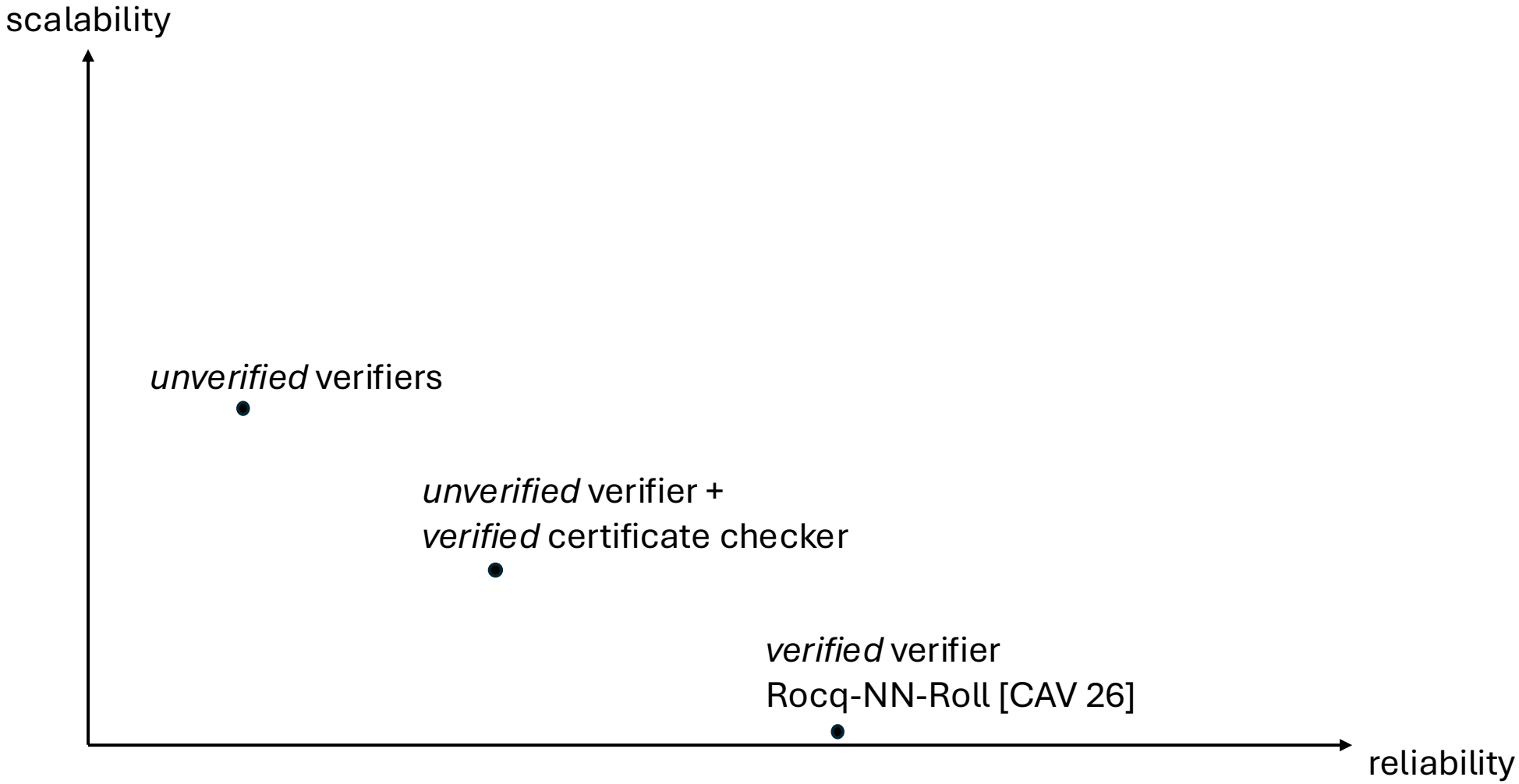
Implementation gap:

- many representations (mathematical object, ONNX, compiler IR, …)
- numerical mismatch between model and digital artifact
- idealized execution semantics

Verifier gap:

- model-conversion defects
- heuristics comprising algorithmic soundness
- implementation bugs
- no transparency of scope of formal guarantees

Landscape: Neural Network Verification

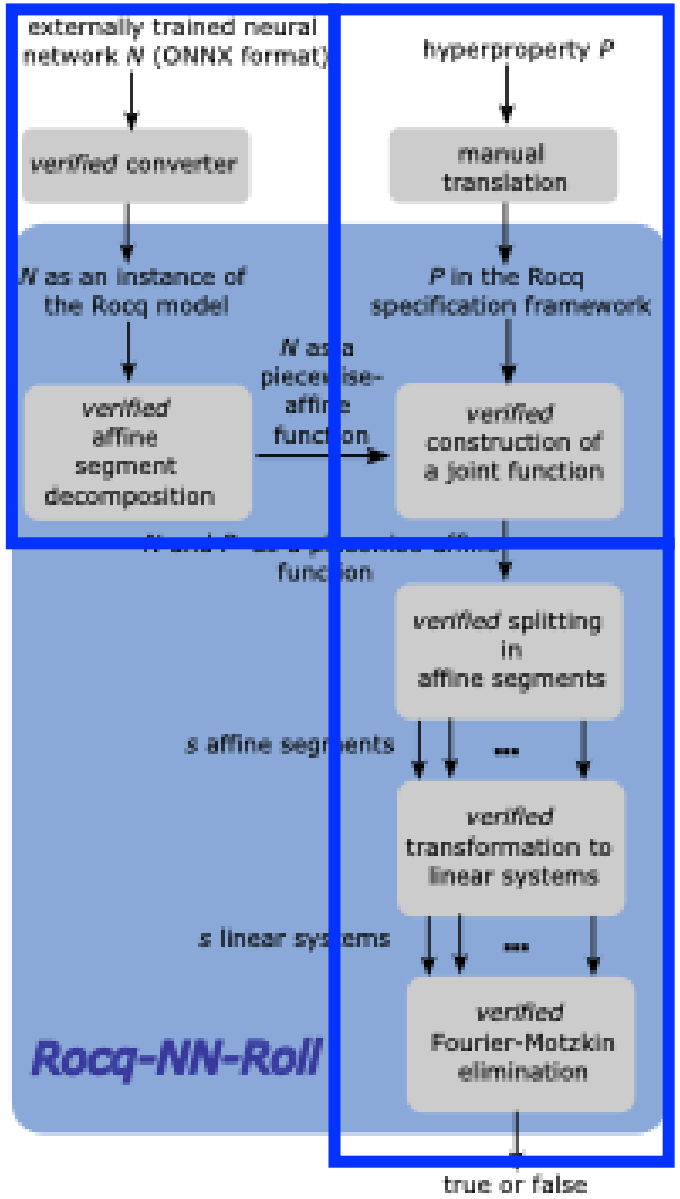


Rocq-NN-Roll: The First Verified Verifier [CAV 26]

[TASE 25]

[NFM 23]

the model

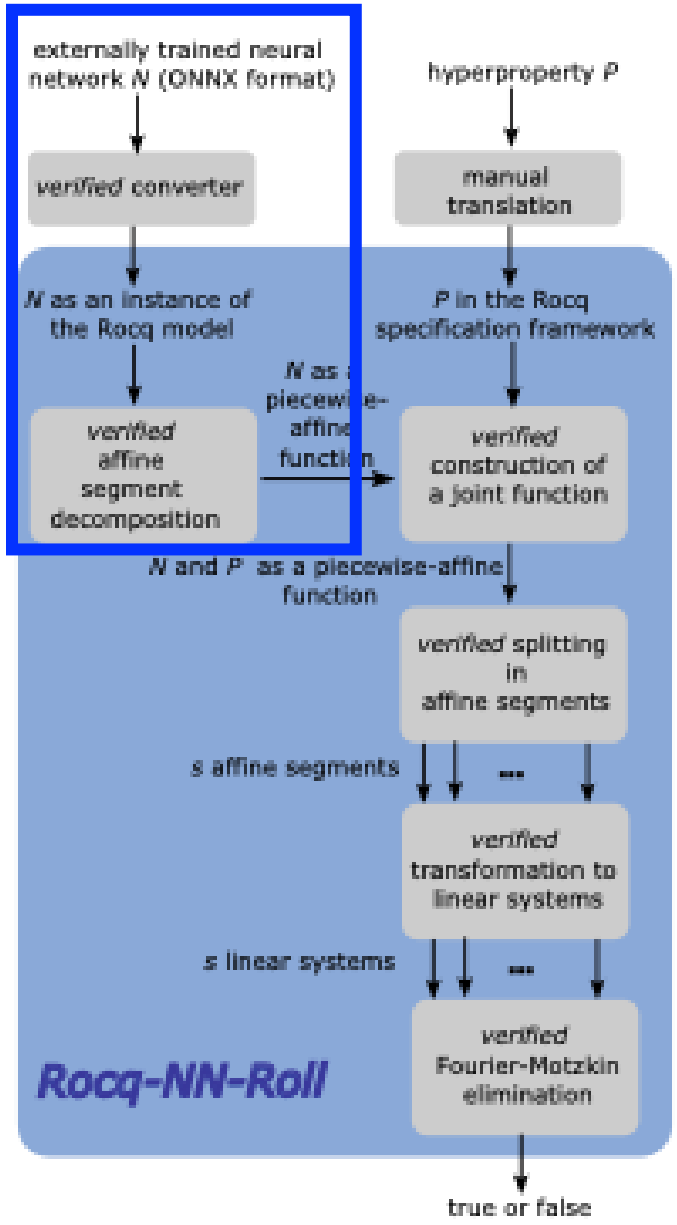


the specification

the verification method

The Model:

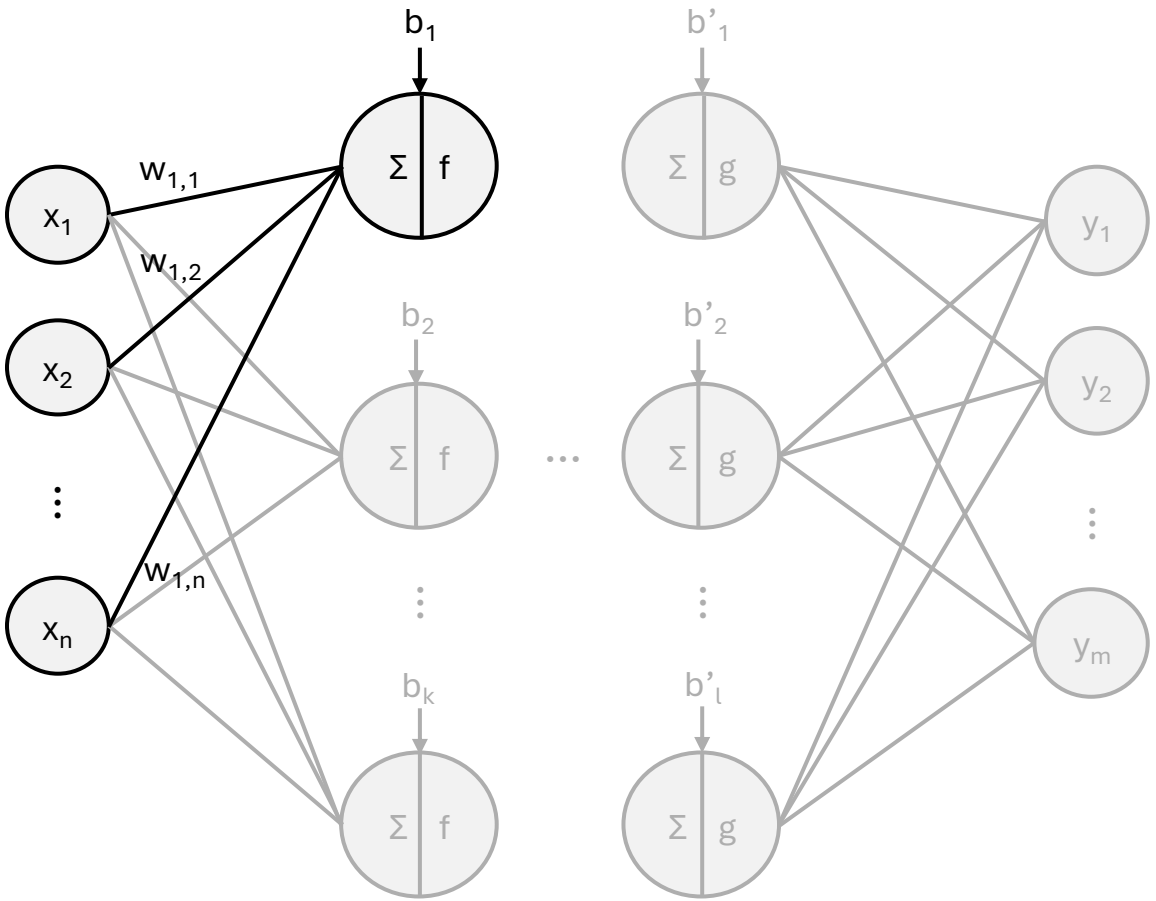
Feed-Forward Piecewise-Affine Neural Networks



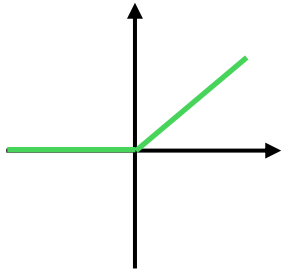
Feed-Forward Neural Networks

neuron: $f(w_1^T x + b)$

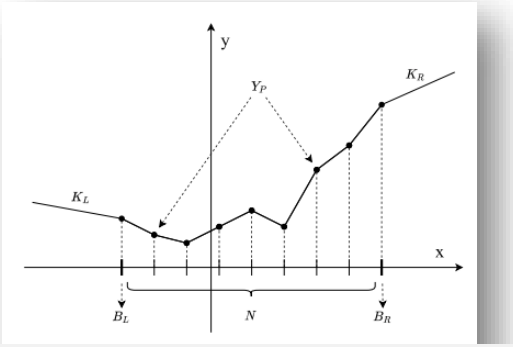
piecewise-affine activation function



examples:



$$\mathrm{ReLU}(x) := \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$



PLU

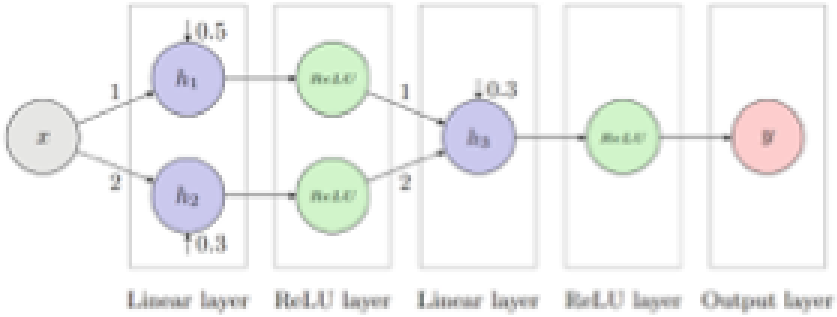
https://ieeexplore.ieee.org/abstract/document/10151955

Feed-Forward Neural Networks in Rocq

```
Inductive TPWANNSequential {input_dim output_dim: nat} :=
| NNOutput : TPWANNSequential
| NNTPWALayer {hidden_dim: nat}:
   TPWAF (RSOAM:=RSOAM) (in_dim:=input_dim)
(out_dim:=hidden_dim)
   -> TPWANNSequential (input_dim:=hidden_dim)
(output_dim:=output_dim)
   -> TPWANNSequential.
```

We want to use that
a piecewise-affine network
defines a piecewise-affine function.

```
Definition example_nn1 :=
   (NNLinear example1_weights1 example1_biases1
   (NNReLU
   (NNLinear example1_weights2 example1_biases2
   (NNReLU
   (NNOutput (output_dim:=1))))))).
```



Polyhedra and PWA Functions in Rocq

```
Inductive LinearConstraint (dim: nat) : Type :=          (* c · x ≤ b *)
 | Constraint (c: colvec dim) (b: R).

Inductive ConvexPolyhedron (dim: nat) :=          (* h-representation*)
 | Polyhedron (constraints: list (LinearConstraint dim)).

Inductive AffineFunction (in_dim out_dim: nat) :=   (* C·x + b *)
 | Affine (C: matrix out_dim in_dim) (b: colvec out_dim).

Inductive AffineSegment (in_dim out_dim: nat) :=    (* polyhedron and affine function (P, f) *)
 | Segment (p: ConvexPolyhedron in_dim) (f: AffineFunction in_dim out_dim).

Record PWAF {in_dim out_dim} := mkPLF {
 body : list (AffineSegment in_dim out_dim);
  prop : pwaf_univalence body}.
```

non-strict

list of finitely many segments (P_i, f_i)

proof-carrying construction

Concatenation of two affine segments

We want concatenation of PWA functions: $(f \oplus g)((x^f\ x^g)^T) := (f(x^f)\ g(x^g))^T$

Given: $S_1 := (P_1, f_1), S_2 := (P_2, f_2)$

Construction for polyhedra $P_1 \oplus P_2$:

Let $P_1 = \{ x \mid \bigwedge_{i=1}^{m} c_{i,1}^T x \leq b_{i,1} \}$ and $P_2 = \{ x \mid \bigwedge_{i=1}^{n} c_{i,2}^T x \leq b_{i,2} \}$:

$$P_1 \oplus P_2 := \{ x \mid \left(\bigwedge_{i=1}^{m} \begin{bmatrix} c_{i,1} \\ \vec{0} \end{bmatrix}^T x \leq \begin{bmatrix} b_{i,1} \\ \vec{0} \end{bmatrix} \right) \wedge \left(\bigwedge_{i=1}^{n} \begin{bmatrix} \vec{0} \\ c_{i,2} \end{bmatrix}^T x \leq \begin{bmatrix} \vec{0} \\ b_{i,2} \end{bmatrix} \right) \}$$

Full construction: $S_1 \oplus S_2 := (P_1 \oplus P_2, f_1 \oplus f_2)$

Composition of two affine segments

We want composition of PWA functions: $(\mathrm{g} \circ \mathrm{f}) := \mathrm{g}(\mathrm{f}(\mathrm{x}))$

Given: $\mathrm{S}_1 := (P_1, f_1), \mathrm{S}_2 := (P_2, f_2)$

Preimage of affine function $f = Ax + b$ on a polyhedron $\mathrm{P} = \{ x \mid \bigwedge_{i=1}^{m} c_i^T x \leq b_i \}$:

$$f^{-1}(P) := \{ x \mid \bigwedge_{i=1}^{m} \left(c_i^T A\right) x \leq b_i - (c_i^T b) \}$$

Full construction: $\mathrm{S}_1 \circ \mathrm{S}_2 := (f_2^{-1}(P_1) \cap P_2, f_1 \circ f_2)$

Affine Segment Decomposition

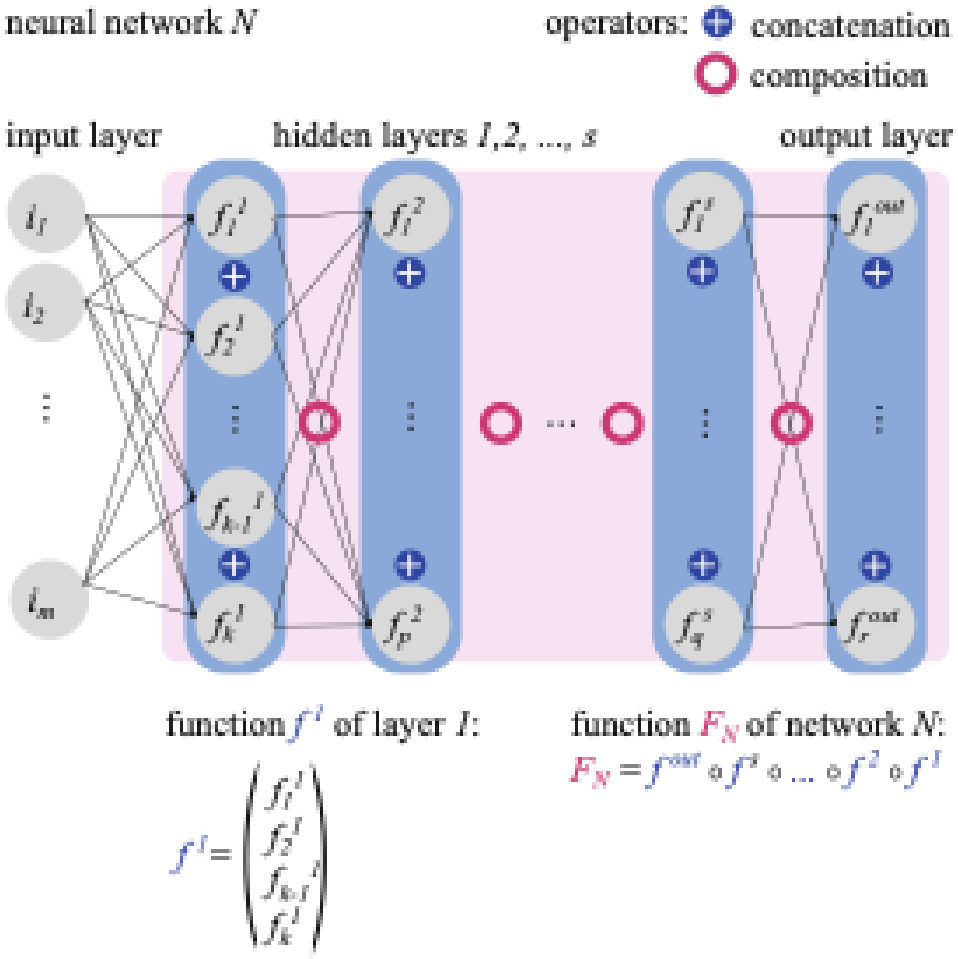
piecewise-affine
neural network

↓

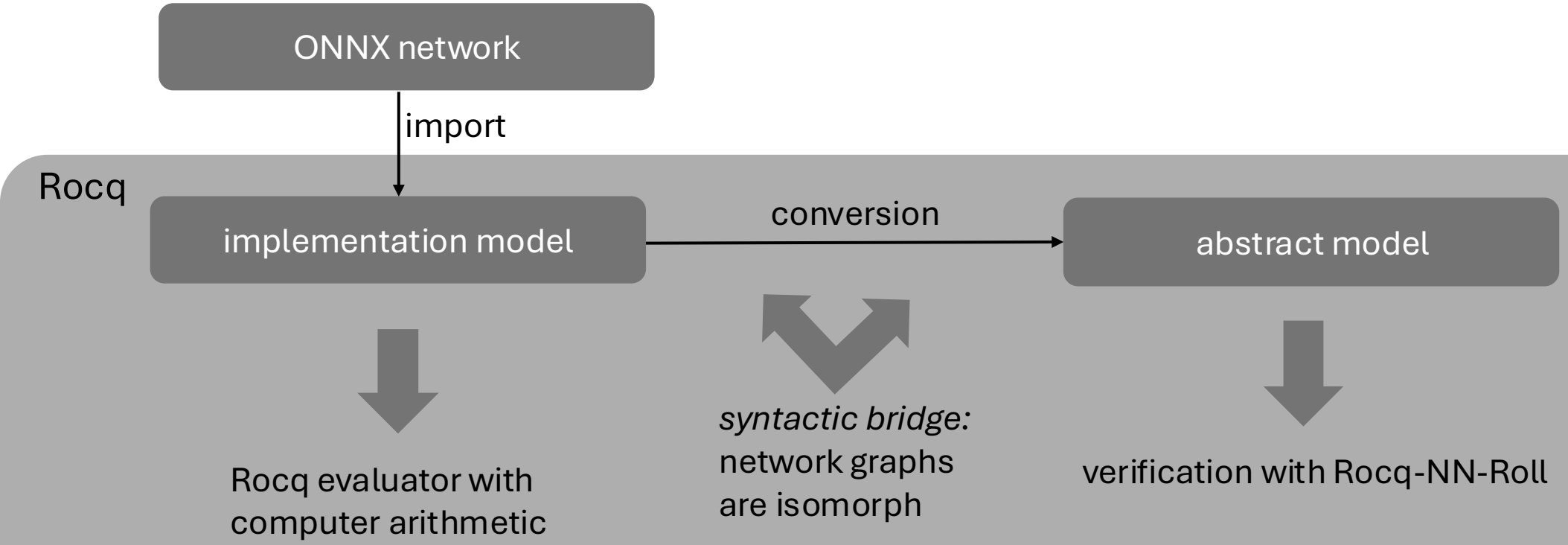
verified
affine segment decomposition

↓

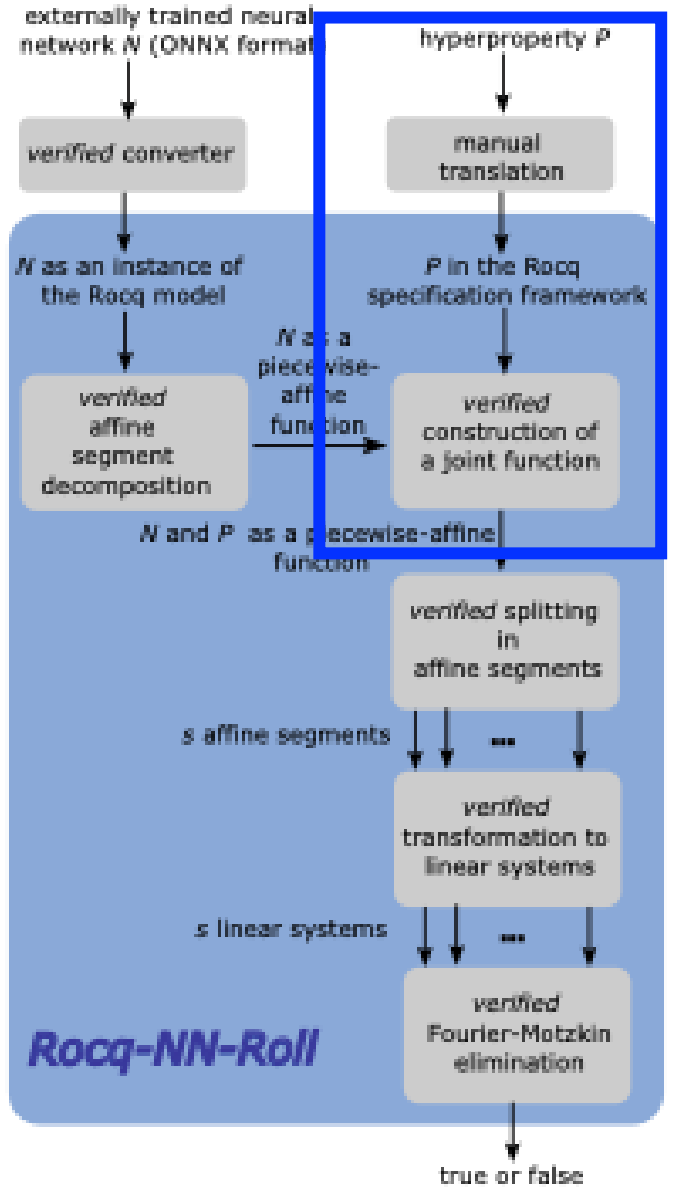
piecewise-affine function



Dual-Embedding Architecture



The Specification:
Neural Network Hyperproperties



Neural Network Hyperproperties [Clarkson & Schneider, ‘10] [Boetius & Leue, ‘23]

traditional specification for all i: $X(i) \implies Y(i, N(i))$

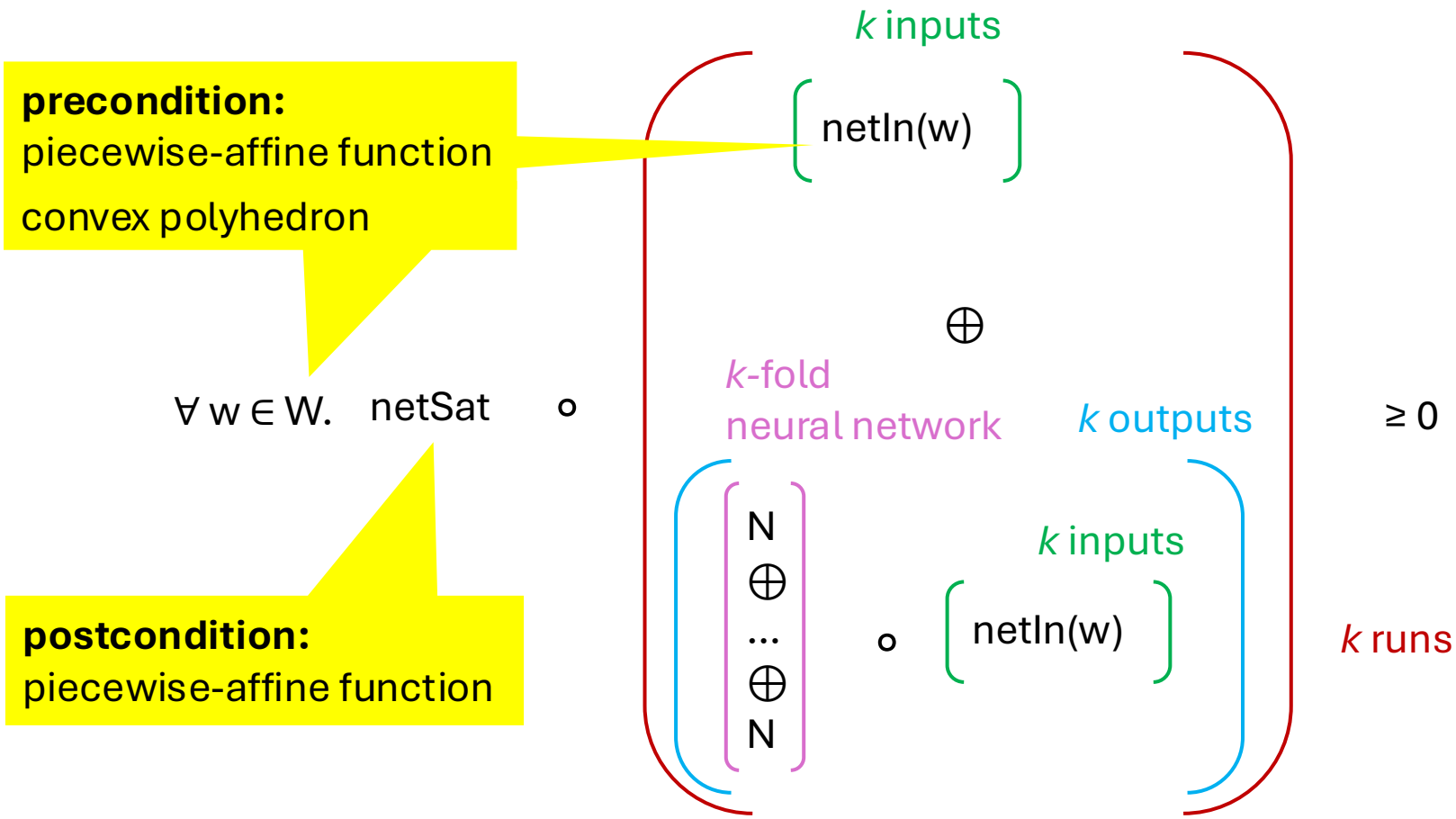
properties on *multiple* runs: robustness, monotonicity

class: *k*-safety hyperproperties

for all $i_1,\ldots, i_k$: $(i_1,\ldots, i_k) \in X \implies (i_1,\ldots, i_k, N(i_1),\ldots, N(i_k)) \in Y$

reduction method: *k*- fold self-composition

Rocq-NN-Roll: Neural Network Hyperproperties



Example: Monotonicity

network: $\mathrm{N}\colon \mathbb{R} \to \mathbb{R}$

monotonicity: $\forall x_1 x_2 \in \mathbb{R}, x_1 \leq x_2 \Rightarrow \mathrm{N}(x_1) \leq \mathrm{N}(x_2)$

2-safety hyperproperty:

$\mathrm{W} = \{(x_1, x_2) \in \mathbb{R}^2 | x_1 \leq x_2\}$

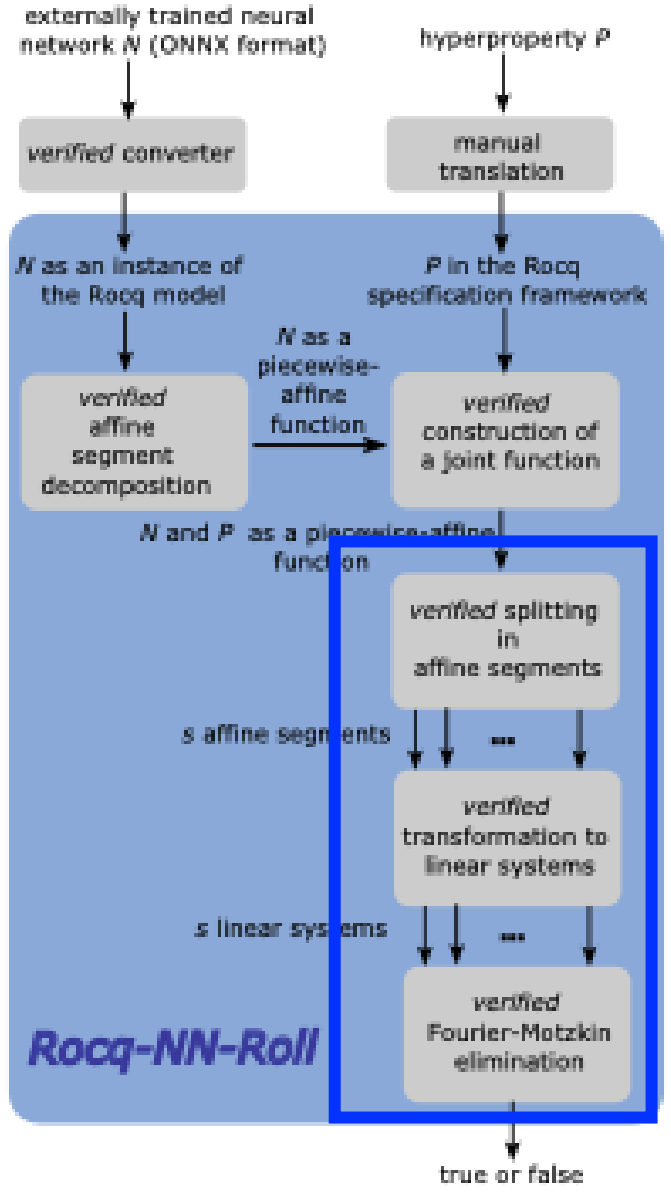
$1 \cdot x_1 + (-1) \cdot x_2 + 0 \leq 0$

$\mathrm{netIn} = \mathrm{id}_{\mathbb{R}2}$

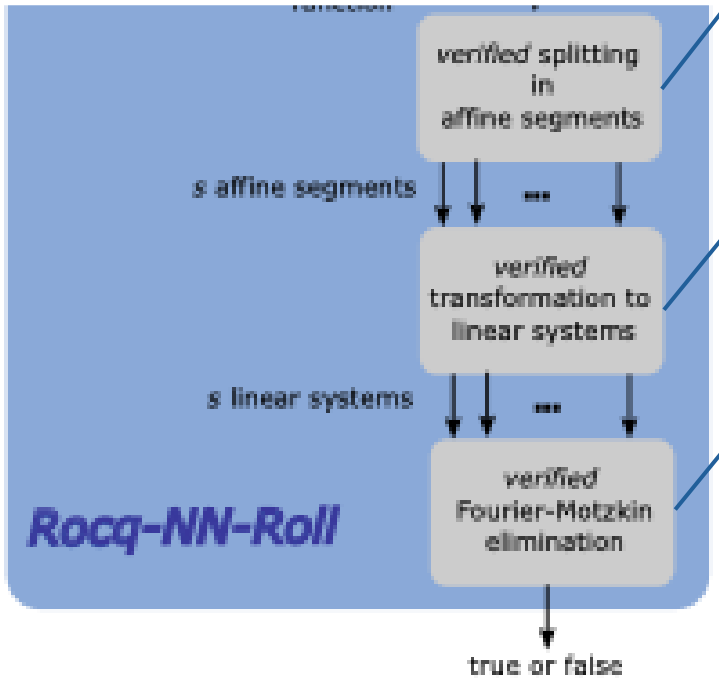
$\mathrm{netSat}(x_1, x_2, \mathrm{y}_1, \mathrm{y}_2) = \mathrm{y}_1 - \mathrm{y}_2$

$0 \cdot x_1 + 0 \cdot x_2 + (-1) \cdot \mathrm{N}(x_1) + 1 \cdot \mathrm{N}(x_2) \leq 0$

The Verification Method:
Linear Feasibility



Verification



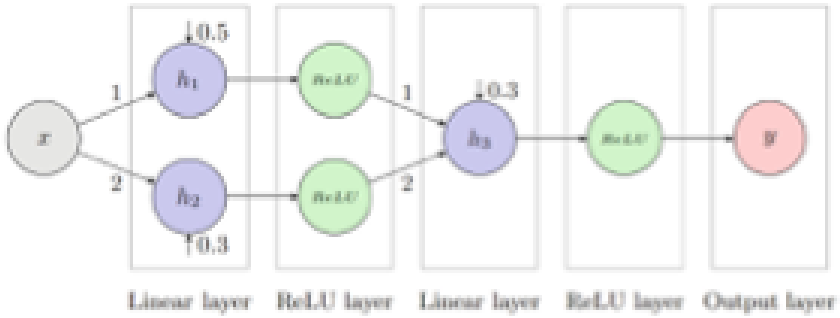
hyperproperty decidable over affine segments of joint function

decision reduced to solving linear systems

customized solver of around 3000 loc

- formal end-to-end soundness proof
- formal end-to-end completeness proof

Example: Monotonous Network



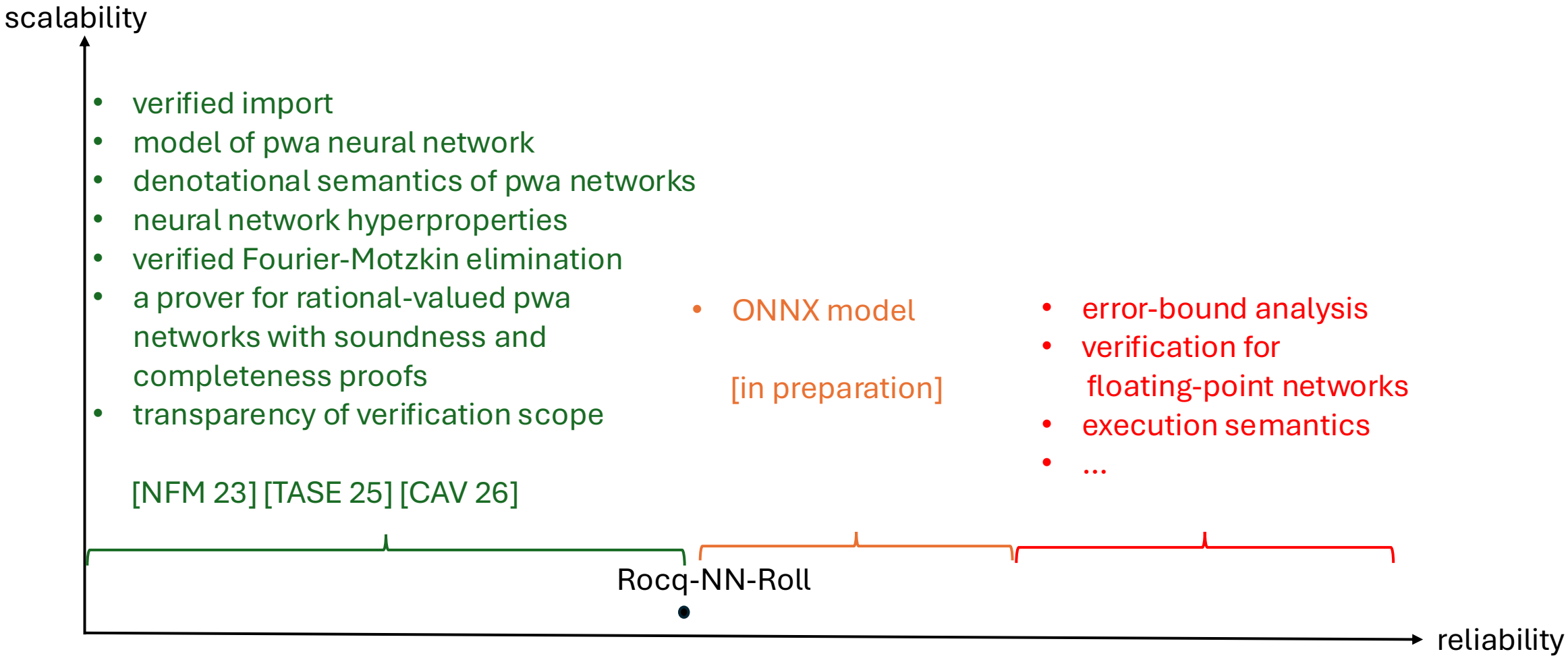

```
Theorem example1_monotone:
  is_monotone_1d example_nn1.
Proof.
  rewrite monotonicity_1d_correct.
  rewrite <- verify_hyperporperty_correct.
  vm_compute; reflexivity.
Qed.
```

Example: Global l_∞ Robustness


```
Theorem example_robust2 [...]:
  is_robust_1d example_nn (toQDEP (0.096)%Q) (toQDEP (0.1)%Q) H1 H2.
Proof.
 apply is_robust_1d_verification.
 vm_compute.
 reflexivity.
Qed.
```

```
Theorem example_not_robust2 [...]:
 ~ is_robust_1d example_nn (toQDEP (0.0959)%Q) (toQDEP (0.1)%Q) H1 H2.
Proof.
 intro Hcontra.
 apply is_robust_1d_verification in Hcontra.
 vm_compute in Hcontra.
 discriminate.
Qed.
```

Rocq-NN-Roll: Soundness Gap



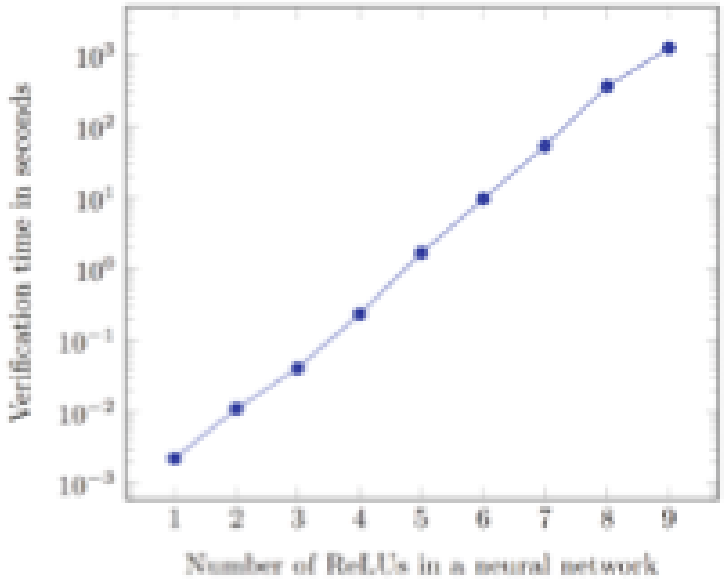
Performance

catastrophic runtime

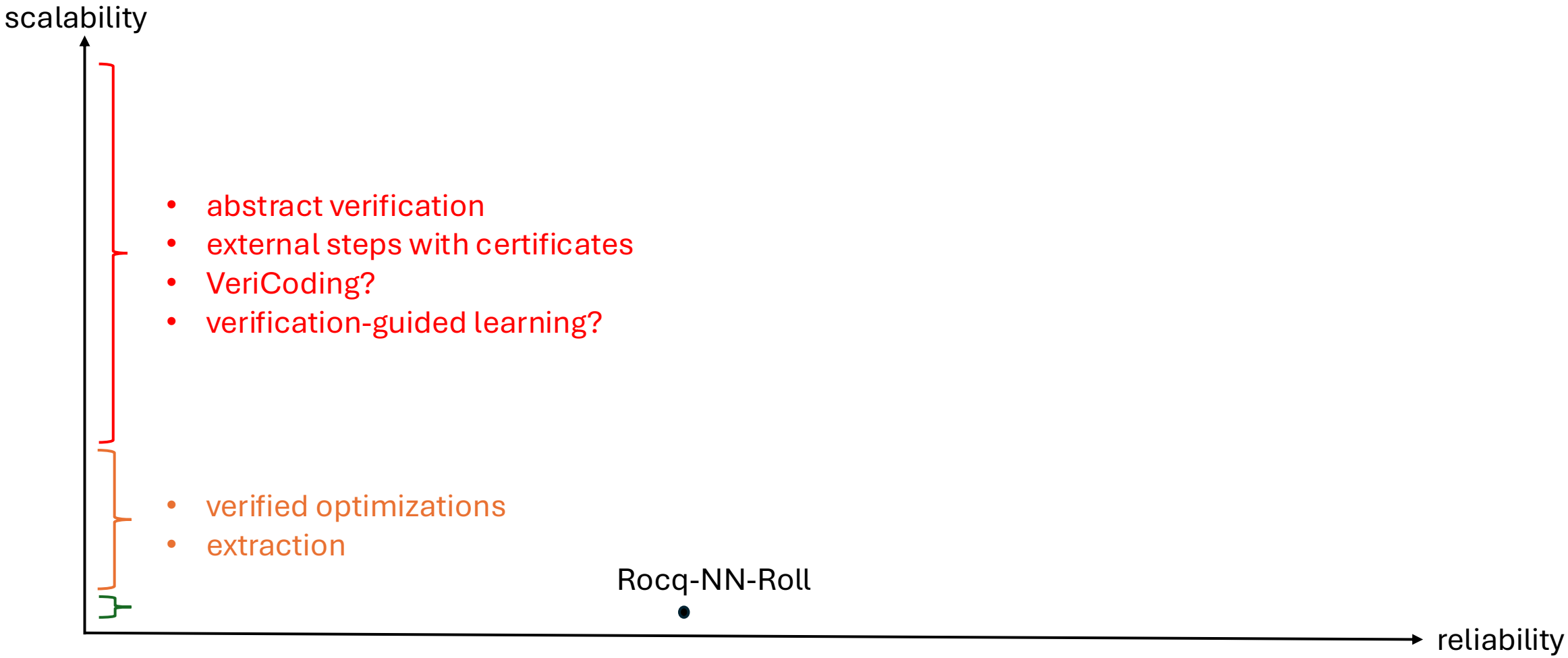
- concatenation & composition are quadratic in the number of segments
- for n ReLUs, repeated application: $O(2^n)$ segments
- for k-hyperproperty:
  - $O(2^{nk})$ segments/linear systems
  - each of size O(kn)
- Fourier-Motzkin elimination is worst-case exponential

experimental evaluation:

- vm_compute
- no extraction



Rocq-NN-Roll: Scalability

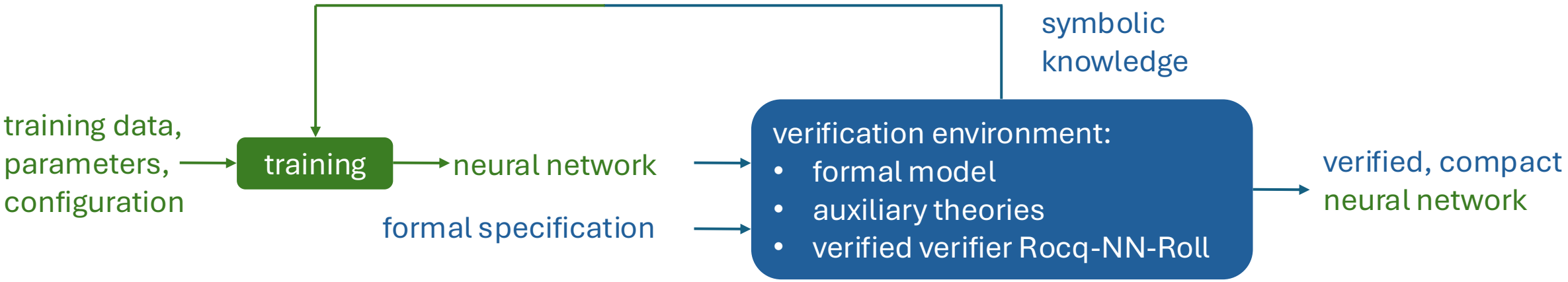


Vision

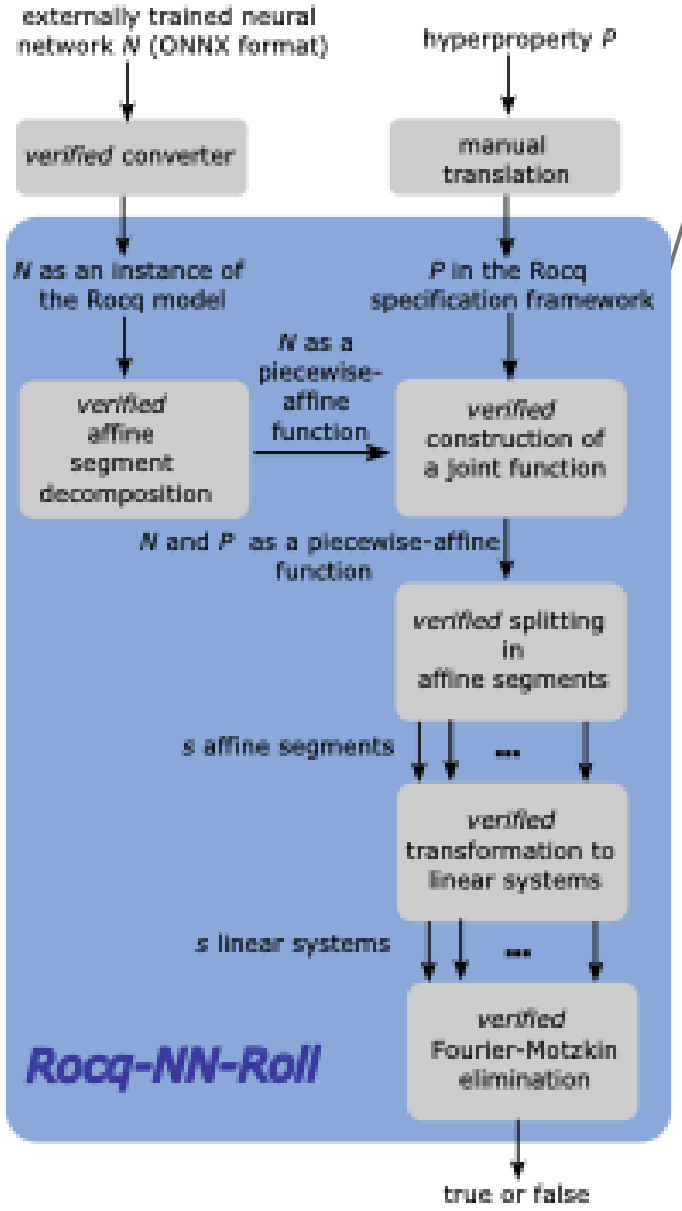
VeriCoding:

- AI synthesized optimizations
- proof assistants offer an ideal environment

Verification-guided training:



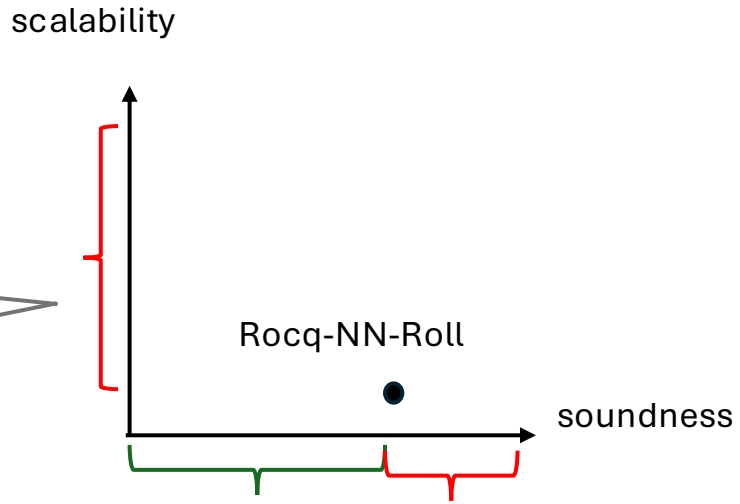
Conclusion



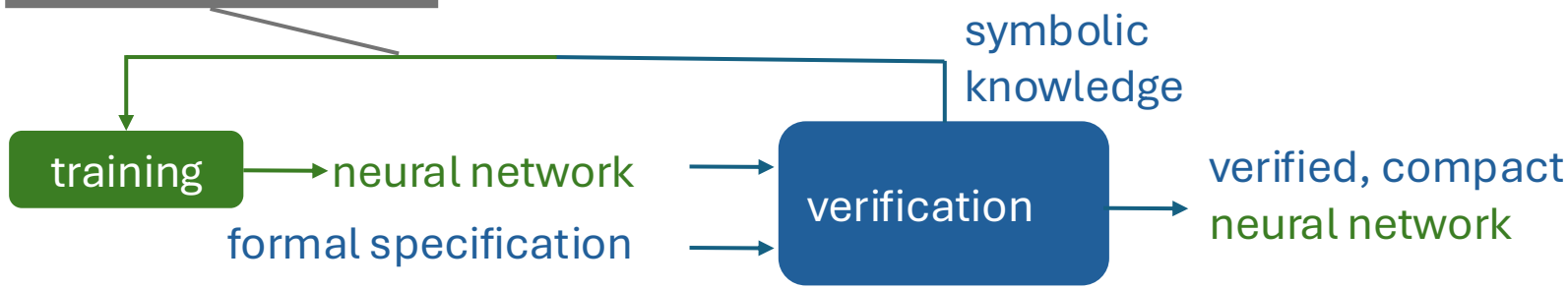
first *verified* verifier

narrows soundness gap

future work: sound mature verifier



vision: verifiable, compact networks



collaboration?

andrei.aleksandrov@fokus.fraunhofer.de
voellinger@tu-berlin.de